베이스라인 설계

생성일	2026년 5월 7일 오후 4:12
태그	

🚀 DMS 벤치마킹 베이스라인 설계도 (Lite Version)

Phase 1: 내비게이션 점수판 만들기 (Promising Matrix)

- **개념:** '유망 점수'
- **설계:** 복잡한 시간 계산을 할 필요 없이, 모든 주문 간의 `[이동 거리 + (대기 시간 페널티)]` 를 미리 계산해서 2차원 행렬로 만들어 둡니다. 앞으로 누구를 묶을지 고민될 때는 무조건 이 점수판만 보고 결정합니다.

phase1_promising.hpp

Phase 2: 초간단 초기 해 생성 (Greedy Constructor)

- **개념:** 일단 모든 주문이 배달되도록 억지로라도 묶어버립니다.
- **설계:** 첫 주문을 잡고, 1단계에서 만든 점수판을 보고 가장 점수가 좋은(가까운) 주문을 차례대로 쑤셔 넣습니다. 용량이나 시간 제약에 걸리면 바로 다음 배달원(새 번들)으로 넘깁니다.

phase2_init.hpp

Phase 3: 미니 ALNS 엔진 (Destroy & Repair)

- **개념:** 만들어진 번들을 흔들어서 더 좋은 경로를 찾습니다.
- **설계 (Destroy):** 복잡한 방법 다 빼고 딱 두 가지만 씁니다.

1. `Random Removal`: 그냥 무작위로 몇 개 뺍니다 (다양성 확보).
2. `Worst Removal`: 비용을 가장 많이 잡아먹는 애들만 뺍니다.

- **설계 (Repair):** 뺀 주문들을 다시 넣을 때, 모든 곳을 다 계산하면 $O(K^3)$이 되어 터집니다. 1단계의 점수판을 보고 "가장 점수가 좋은 상위 5개의 번들"에만 실제로 찔러 넣어보고(Greedy하게) 가장 싼 곳에 확정합니다.

phase3_alns.hpp

Phase 4: 배달원 스와핑 (Rider Optimization)

- **개념:** 경로는 그대로 두고 가성비를 챙깁니다.
- **설계:** Phase 3가 끝날 때마다, 현재 묶여있는 번들을 도보, 오토바이, 차량으로 각각 돌려보고 **가장 싼 배달원으로 명찰만 바꿔치기** 합니다. 코딩은 제일 쉬운데 점수 방어에 엄청나게 유리합니다.

phase4_rider.hpp

Phase 5: 퍼즐 맞추기 (Set Partitioning with Solver)

- **개념:** 루프를 돌면서 발견한 좋은 번들들을 모아서 최종 제출용 조합을 짭니다.
- **설계:** Phase 3~4를 도는 동안 괜찮았던 번들들을 전부 리스트에 저장해 둡니다. 루프가 끝나면 이 리스트를 솔버(**Gurobi**)에 밀어 넣고 중복 없는 최저가 조합을 뽑아냅니다.

phase5_solver.py

🏁 1. 준비 단계 (단 한 번만 실행)

알고리즘이 켜지면 제일 먼저 딱 한 번만 수행하고 끝나는 작업들입니다.

- **Phase 1 (점수판 생성):** 모든 주문 사이의 '유망 점수(Promising Matrix)'를 계산해서 메모리에 올려둡니다.

- **Phase 2 (초기 해 생성):** 점수판을 보고 대충 스윽스윽 끼워 넣어서 "일단 모든 주문이 배달 가능한 첫 번째 시간표"를 만듭니다.

🔄 2. 탐색 루프 (시간이 끝날 때까지 반복)

이제 제한 시간이 끝날 때까지 아래의 과정을 계속 반복합니다.
여기서 루프는 '작은 루프'와 '큰 루프'로 나뉩니다.

[작은 루프 (수만 번 반복) : 좋은 번들 찍어내기]

- **Phase 3 (ALNS):** 쪼개고 다시 합치면서 새로운 경로를 만듭니다.
- **Phase 4 (배달원 교체):** 방금 만든 경로의 배달원을 더 싼 사람으로 바꿉니다.
- *이 작은 루프를 수백~수천 번 반복하면서, 그동안 발견한 '가성비 좋은 번들(경로)'들을 몽땅 리스트(창고)에 저장해 둡니다.*

[큰 루프 (주기적으로 실행) : 드림팀 결성]

- **Phase 5 (솔버 퍼즐 맞추기):** 작은 루프가 어느 정도 돌아서 창고에 좋은 번들이 꽉 차면, 무거운 Gurobi Solver를 한 번 훅 돌립니다. 창고에 있는 번들들을 조합해서 "현재까지의 최저가 조합(Best Solution)"을 뽑아냅니다.
- **피드백 (핵심!):** 솔버가 찾아낸 이 최저가 조합을 **다시 Phase 3의 출발점**으로 던져줍니다. "이게 지금까지 찾은 제일 좋은 조합이니까, 여기서부터 다시 쪼개고 합쳐봐!" 하는 거죠.

💡 요약

[Phase 1] ➔ [Phase 2] ➔ 🔁 큰 루프 시작 (🔁 작은 루프 [Phase 3 ➔ Phase 4] 반복 ➔ [Phase 5])
➔ 최종 정답 제출

개발 환경과 도구(Tech Stack): C++/ Python

main.cpp

`<chrono>` **타이머:** 파이썬의 `time.time()`과 똑같은 역할.
(10초 제한이라면 알고리즘이 결과를 출력할 여유 시간 0.5초를 남겨두고 **9.5초**에 루프를 딱 끊고 나오도록 설계했음.)